

Lec 57 Transformer Architecture: Encoder

Lec 57 Transformer Architecture: Encoder

Lecture	57 / 63 + intro
Duration	39:35
Captions	English (auto-generated)
Words	5,168
Video	PBBWBo7OL3M
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome back to the lecture series on fundamentals of generative AI and large language models. In today's session, we will discuss about the transformer architecture. In particular, we will discuss about the encoder part of the transformer architecture, okay? So, the contents for today's discussion are motivations for transformer architecture, introduction to transformers, function of encoder and decoder in a transformer, positional encoding, numerical in positional encoding, one numerical example we will do, encoder architecture, residual connections and layer normalization, and position-wise feed forward network. So, these are the contents for today's discussion.

[00:49] So, let us start with the motivation for transformers. So, we have seen in our last lecture also that there are sequence-to-sequence models like RNNs and LSTMs, which were doing to some extent a good job in capturing the long-term dependency. To some extent, they were able to capture the long-term dependency when a sequence of words are given as input to the model. But, there are certain limitations which were faced in RNNs, LSTMs, and GRUs, or any uh sequence-to-sequence models. That means which takes an input sequence and gives an output sequence, okay? So, those

we call as sequence-to-sequence models. So, there were certain limitations of these sequence-to-sequence models like RNNs, LSTMs, and GRUs. So, what are those

[01:47] limitations? First of all, in all these sequence-to-sequence models, the computation is done in a sequential way. That means each word is computed in a sequential manner. Okay? So, first word, then second, then third, then fourth. And due to this computation sequential computation, what was happening was the training process become slower. Okay? Also, there is difficulty in parallelization, which means that it was difficult to parallelize or make multiple operations going together or working operations getting performed simultaneously. So, parallelization was difficult, and sequential computation was making the training very slow. Okay? Also, even though uh LSTMs were able to capture the long-term

[02:43] dependency, but not very very long-term dependency. When when the you know, there are several sentences or there are a lot of words in between uh two words which are you know, having some relationship, there that long-term very very long-term dependencies became a problem. LSTMs were not able to capture that long-term dependency. Okay? So, we have seen with attention mechanism attention mechanism we have uh seen in our last lecture, it improved the sequence-to-sequence models, but uh the main problem there was still it was, you know, having some dependency on the recurrence, which means that even the attention mechanism, that means we are not remembering or giving attention to all the past words equally, but we are giving

[03:40] attention to to the very specific words which have some you know, uh relationship with the next next or next predicted word. Okay? So, that way attention mechanism was able to, you know, solve certain problems, but still this recurrence was there. Recurrence means what? That means there is a hidden state which was keeping memory of the previous word, and then that is taken into account to predict the next word. So, this recurrence was there, okay? So, and whenever there is a recurrence, what happens is the upcoming predicted word depends on the previous word. That previous word depends on the previous, you know, previous word to the previous word. So, there that recurrence was there, and scientists started thinking that is

[04:31] there any possibility that we can remove the recurrence entirely because that will speed up the process a lot. And that is where Vaswani et al. came up with this ground-breaking paper research paper, Attention Is All You Need, in 2017. So, that is the paper where transformers are introduced, and the transformers are based only on attention mechanism, eliminating the recurrence and convolutions all together, okay? So, that was the motivation of transformers because of certain limitations in sequence-to-sequence models, particularly the sequential computation, the difficulty in parallelization. So, all these motivated for the transformer architecture to come into, you know, existence.

[05:25] Now, we will look into what is this transformer architecture, what are the different parts of the transformer architecture, what are the different functions of these different parts, okay? So, the transformer is basically made up of two parts, the encoder and the decoder, okay? So, if we see in the encoder part, there are six encoder layers. Encoder layer 1 2 3 4 5 6. So, they are stacked over each other. So, there are six encoder layers in the original paper Transformer architecture, and these six layers are stacked one over the other, and that makes your entire encoder.

[06:12] Okay? And each of these encoder layers had two sub layers. One is multi-head self-attention layer, another one is the position-wise feed-forward network. Okay? And that is there for I have shown it here for encoder layer one, and I have shown it for encoder layer six. And if you see the dots, which means for every layer, there is these two sub layers. So, encoder layer one have multi-head as well as position-wise feed-forward network.

[06:46] Encoder layer two also have the same two sub layers. Encoder three also same sub layers, and so on and so forth till encoder layer number six. Okay. So, today's discussion is mainly on the encoder part of Transformers. Decoder is something which we will look into in our next lecture. Okay? So, this layer one, layer two, layers till layer six, they have exactly the similar architecture. There is no difference in the architecture of layer one to layer six. All of these layers have two sub layers. One is the multi-head self-attention and the FFN or the feed-forward network, position-wise feed-forward network. Okay?

[07:30] The architecture is exactly same. What is different then? Why do we require? We require the stacking multiple layers actually allows the model to build a deeper understanding of the sentence. Okay, so that stacking is required because it gives a much better understanding of the sentence. And what is different in all these layers? The difference is the weights are different in these layers. The learned weights are different. Okay, so that is uh you know, about the encoder parts overall architecture.

[08:07] Okay. Now, we will go to the function of the encoder and decoder. Till now, we have just understood the transformer is made up of two main components, the encoder and the decoder. We are talking about the encoder architecture in today's session. Okay. So, we will see what is the function that the encoder is supposed to do. What is the function that the decoder is supposed to do to have an overall understanding of the transformer architecture. Okay.

[08:37] So, just take the example of language translation. Suppose I want to translate a sentence from English to German. Okay. In this case, what is the function of the encoder and what is the function of the decoder? The transformer is used for language translation in this case in this example. And I will we will see what is the function of the encoder and decoder. Okay. So, the encoder's work is to understand the input. The input sentence in English.

[09:13] And the decoder's work is to generate the output. That means it will generate the translation in German. Okay. So, that is the function of the encoder and decoder. Okay. Now, if we see the encoder processes all the input tokens simultaneously. Okay. And it generates a contextual representation of every token. So, we have seen you know, there is attention mechanism that goes inside the encoder, which gives a contextual information, you know, of every token in the encoder part. Okay? This Previously, also we have seen. So, the main work of the encoder, it takes all the input tokens simultaneously, not one by one.

[10:06] And then it generates a contextual representation of every token. Whereas, what is the work of the decoder? We know the decoder is generating the output. Now, while generating the output, the decoder doesn't generate all the words at time. During inference time, it generates one word at a time. Okay? So, one word is generated, then the second word gets generated, then the third word word gets generated, and so on and so forth. Okay?

[10:37] So, even though the encoder takes all the words together, the decoder generates words, that means generates the German words. The encoder is taking the English words. The generate The decoder is generating the German translation of those words, and one at a time. Okay? So, at every step, the decoder There are a couple of things the decoder does. It looks at the words it has already generated. It looks at the encoder's output, what is coming from the encoder, basically the contextual representation of every token that is coming from the encoder.

[11:15] And then, using those, it predicts the next word. Okay? So, that is what the encoder and decoder is supposed to do in a transformer architecture. Okay? Now, we will come to some concept called as the positional encoding. It's a new concept. Uh we will see what exactly this positional encoding it is and how it is used in the transformer. Where exactly it is used in transformer. So, from the name itself we can tell or we can guess that it has to do something with the position, position of a token or position of a text, okay?

[11:54] So, if we see other sequence to sequence models like RNNs or LSTMs, words are processed one after the other. The sequence is first word, then second word, then third word, then fourth word. Now, what happens due to this? The order is known to the model. So, say the cat sat on the mat. So, the the came first, then came cat, then sat on the mat. So, the sequence of the tokens that are coming inside the model, that is known to the model.

[12:27] Okay? So, the hidden states carry information from the previous words, so the model naturally knows the order of the words. So, there is no problem with the order of the words or the order of the tokens, okay? But, in the transformer, all words are processed simultaneously, okay? So, when all the words are processed simultaneously in the encoder part, see? What happens is all are having some embedding,

you know, vector embeddings. All these tokens are having some vector embeddings.

[13:01] Now, when all the words are put to the model simultaneously, there is no footprint of the position or the order in which the words are coming in the sentence, okay? So, » [clears throat] » what happens is that order information is lost when you are giving all the tokens together to the model. So, positional encoding injects information about the order of the tokens. So, this is the main purpose of positional encoding.

[13:36] So, it basically injects the information. It keeps the footprints or the fingerprints of the order of the words, okay? And then it is added to the embedding to give the final embedding. Okay? So, that is the work of positional encoding. And there are two equations. These are sinusoidal, you know, positional encoding. So, we have positional encoding. PE means positional encoding. Within bracket $POS\ 2i$, POS means is the short form of position.

[14:13] We will see what is this i . i is basically the index of the word in the sentence. That is equal to $\sin$ of $pos - pos_{pe}$ position divided by 10 to the power of 4 to the power of $2i$ by d . Now, what is this d ? d is nothing but d is the dimension of the model, okay? So, d model. Whereas, positional encoding for the positions of $2i + 1$ is equal to $\cos$ of position divided by 10 to the power of 4 o- all to the power of $2i$ by d . Now, what what does it mean?

[14:56] That means your even dimensions are using the sign and odd dimensions are using the cosine function. Okay? So, these are two equations with which we find out the positional encoding. Now, let us have a numerical example. We will, you know, do a solve a numerical example to uh you a better understanding of how the positional encoding uh works. So, assume our embedding dimension d_{model} equal to four. What exactly I mean by embedding dimension? It means whatever vector embedding I'm getting for a word, that will have four numbers, 1 2 3 4. So, four values will be there. So, that is what is your uh four. Model dimension is four. Now, as I told in the previous slide also, that the original paper have

[16:03] dimension of 512, d_{model} equal to 512. I'm taking it four because if I have to do for 512, the calculation will be huge, and which I don't want to. So, I have taken a small number four, and we will, you know, solve it out here, so that we get a clarity on how these positional encodings are done. Now, let us take the sentence the cat sat on the mat. So, this is my sentence, okay? Now, the positions, which I write as POS, they go like 0 1 2 3 4 and 5.

[16:47] Okay? So, if I tell position equal to two, I'm trying to find out that uh you know, it I'm just two means this is a sat word. So, I'm trying to find out positional encoding for the word sat. A word or token, okay? Word or token. So, POS equal to two, that means I'm trying to find out. So, trying to find positional encoding for the word or token sat, okay? Now, we will just do the calculations for this one. So,

positional encoding, which is your PE position. In this case, position equal to two $2i$.

[17:41] That is equal to sine of POS position divided by 10 to the power of four to the power of $2i$ by d_{model} . So, that was the equation which we have written in the previous slide also. And the other equation also, I'll write it here. PE positional encoding for the position. In this case, it is two $2i + 1$, which is your odd place. So, you know, odd place in this uh vector, okay? Equal to cos of position divided by 10 to the power of four $2i$ by d .

[18:24] Okay? Now, for i equal to one, let us solve it. For i equal to zero, let us solve it, okay? So, in that case, positional encoding for two two is coming because your position is two here. I have already taken. And zero comes here because $2i$ equal to zero for i equal to zero, okay? That is equal to sine of two by 10 to the power of four to the power of zero. Which is nothing but sine two. And its value is 0.9093. If there is any typo in my writing, please, you know, you can correct it by yourself or bring it to my notice.

[19:09] And for the positional encoding for the second position for $2i + 1$ for $2i + 1$ it will be 1 here. That is equal to cos of 2 by 1 2 3 4 to the power of 0 which is equal to cos 2 equal to -0.41 61. So, what we have got you know we are what we have solved here is for i equal to 0. We have got the sign and the cos values of the positional encoding values that will go in the zero zeroeth and first place. Okay, so these two values are done actually.

[19:53] Okay? Then we will go to i equal to 1. What do I get for that i equal to 1 positional encoding POS position is 2 for i equal to 1 $2i$ is 2 equal to sine of 1 2 3 4 to the power of 2 by 4 which is basically your half. That is si- sorry sine of some Once again, I'll write this one. There is some mistake in the writing part. I'll write it once more so that it becomes clear. So, it is equal to sine of 2 divided by 2 3 4 2 by 4 which is equal to half.

[20:59] That is equal to sine 2 by 100 which is sine 0.02 and it is equal to 0.0200. Okay, then positional encoding of two, this is for the even uh sorry, odd um you know, place. That is $2i + 1$ equal to cos of 2 by 1 2 3 4 2 by 4, which is once again half. That is sign of Sorry, this is cos, not sign. This is cos of 2 by 100 equal to cos of 0.02, which is almost equal to 0.9998. Okay. Now, I have got the positional encoding for the second position.

[22:12] What is it? It is a vector consisting of all the numbers that I have got. 9093 minus 0.4161 0.0200 0.9998. Okay. So, this is the positional encoding for the word that is present in the position two, which is nothing but my sat. So, this is the positional encoding. Now, if your vector embedding if vector embedding for sat is given by $X_1 X_1 X_2 X_3 X_4$, then your final embedding will come as whatever values you have got here plus X_1 , so 0.9093 plus X_1 , then minus 0.4161 plus X_2 uh 0.0200 plus X_3 and then 0.9998 plus X_4 . So, this is your final embedding.

[23:41] Which goes into the transformer architecture. Okay? So, this is about the positional encoding. Okay? I hope it is clear now to all of you. Okay? Now, we will have a look into the encoder architecture. What are the things that are there inside the encoder architecture? What is the What Why What is the significance of each and every block inside the encoder architecture? Okay? So, we will start with the input, which is your input sequence of words, and then this input sequence is changed into input embeddings, which are the vector embeddings for the words.

[24:16] And these, as we all know that these input embedding convert the tokens or the text into continuous vectors of fixed dimension. In this case, the model equal to 512, which is used in the original paper. Okay? Now, with the input embeddings, we add the positional encoding also, which bears the footprint of the exact place where that word is coming and sitting. Okay? So, the cat sat on the mat. So, the mat is at the end of the sentence. So, it will So, it will have that finger footprint that it is at the end of the sentence, not at the beginning. So, that is the function of your positional encoding. And now, what happens is this positional encoding and the input embeddings, they are joined together to give you the final

[25:04] embedding that goes inside the um encoder, okay? So, this positional encoding provides information about the relative or absolute position of the tokens in the sequence of words, okay? Then this input goes to multi-head self-attention. We all know what is the function of multi-head self-attention. Multi-head self-attention actually looks into a sequence of word or a sentence from various perspectives, trying to understand various kind of relationships that can be there in that sentence. Like when I tell various kind of uh relationships, it can be the semantic relationship, it can be the syntactic relationship, it can be the structural relationship as well. So, more number of heads are there, more, you know, the

[25:51] understanding of the sentence from various perspective becomes much more richer, okay? And um that is the function of the multi-head self-attention. So, attends to different parts of the same input sequence simultaneously, capturing multiple contextual relationships in parallel. So, parallelly it is finding out the relationship between words in a sentence, okay? And then there is add and norm operation. Add is basically coming from the residual connection where the original input is getting added to the output of a sublayer. And then there is a normalization function that is working above this add and norm. And then what is the function of this one? It stabilizes the training process, okay?

[26:35] So, that is the function of this add and norm. And then after that we have this position-wise feed-forward network. Now, what is this What is the function of position-wise feed-forward network? It basically learns richer and complex feature representations after the attention. So, attention has, you know, we have taken care of the attention mechanism in the multi-head self-attention block. And then after the add a norm layer, it is going to a position-wise feedforward network. So, it is basically

a feedforward network which Why it is used? Because it will learn much more complex features representations in the uh sentence after the attention mechanism has uh you know, operated on that one.

[27:20] Then again, there is an add a norm which wraps around from here to here. And then that's all. So, that is the entire this place which is marked in red, that is your encoder part. And there is an NX written here, that means n number of times it can be multiplied. So, basically these encoders come in stacks. So, if it is n equal to six, that means there are six encoder layers which are coming one after the other. Okay? So, this is the architecture and these are the blocks that are there inside the encoder architecture.

[27:56] Done. Now, we will look into the residual connection and layer normalization. What exactly we are doing there? So, you look into this picture. X is my input which is the word embedding with positional encoding. Okay? So, that is my input. And this goes to a sublayer. The sublayer is can be, you know, uh the multi-head attention layer. The sublayer can be, you know, here in this case, the input comes actually to the multi-head. So, this is the multi-head self-attention layer.

[28:37] Okay. Now, whatever is the output of this sublayer, this output this sublayer output output of the sublayer is written as sublayer X. That means it is taking input as X and then performing some operations which I'm writing as and the output that is coming here is sublayer X, okay? And then I am adding this X here. So, this is the addition operation that is happening with the with your residual connection, okay? And then we are taking normalization. So, layer norm comes after we have added the original input and original input is added to what? It is the output of the sublayer operation, okay? Now, why do we need to add the original input? Because lot of times what happens is the operation that is happening in

[29:36] the sublayer, it changes the representation fully, okay? So, what happens in this case is if the representation is changed fully, then important information may disappear. To keep that information intact, the transformers preserve the original representation using the residual connection. So, whatever happens in the sublayer after operation in the sublayer, you still have this X added so that your important information from the input is not getting lost, okay? So, that is the reason for residual connection. And what about the layer normalization? Why do we require the layer normalization? Normalization we all know that when the output of different neurons are of different scales, varying scales, we require a

[30:26] normalization process, correct? And what does this achieve, the normalization process? It stabilizes the activations and improves the optimization. Basically, it stabilizes the training process, makes the training faster, okay? So, that is the reason for the layer normalization. Then, we will look into the position-wise feedforward network. Okay? So, uh what is this feedforward network? What exactly it is doing?

So, we have I have kept this picture once again here. And uh Okay?

[31:05] So, this is position-wise feedforward network here. And if we see the embeddings input sequence embeddings join, you know, they are added with the positional encoding. It goes to multi-head self-attention, then add a norm, then it goes to FFNN. Okay? So, position-wise uh wise feedforward network comes after multi-head self-attention. In between, there is a add and normalize function that is happening. Now, what is the from the multi-head self-attention, that it does we have gathered all the information from relevant perspectives of the sentence.

[31:42] Okay? Then, the FFNN layer or the feedforward network, it transforms independently using each token is transformed independently using the same feedforward network. What does it mean? That means each of these tokens of the input sentence, they are going to individual feedforward networks. And what is the function of the feedforward networks? As we have already seen, it gives a better understanding or better contextual representation, much richer representation, that means much better understanding of the input sequence.

[32:20] Okay? So, we'll just have a look what exactly I mean by position-wise FFNN. So, if W_1 to $W_{\text{capital T}}$ are the words in the input sequence, then what happens here is we get the embeddings from them, and then they are added with the positional encoding to give the final uh input embeddings. And these input embeddings go to the self multi-head self attention. And there there are separate FFNs feed forward networks for each and every token. So, that is the reason they are called position-wise feed forward network. Okay. So, that is about the position-wise feed forward network. And with this one okay, we will see what is there inside the feed forward network. Okay. So, we have seen there is position-wise feed

[33:15] forward network. Every individual tokens are getting to go through individual FFN. Okay. Now, what is there inside the FFN? It consists of two linear transformations and one ReLU activation between two linear transformations. So, how it is? So, if your input is having the dimension 512, well, I have taken it from the original paper. Then, the linear transformation linear transformation, it changes 512 to 2048 dimensions. Okay. So, there is an expansion of the space. Okay. Dimension.

[33:50] And then it goes to the ReLU activation function. And there is another linear transformation which again brings back the you know, expanded dimension to the actual dimension. Okay. So, that is the architecture of a feed forward network. Inside the feed forward network, what is there? Now, from that linear transformation, we get the output. Okay. Now, there is an equation which gives uh what is the output from this feed forward network. We will we will look into it that $XW_1 + B_1$.

[34:26] X is your input. Okay. $W_1 + B_1$ W_1 are the weights of the first transformation and B_1 is the bias. So, $XW_1 + B_1$. So, this is coming from the first linear transformation. Then, it is going through ReLU. So, this ReLU gives of zero and the

output. So, that is where your max comes into picture and the entire thing is now then multiplied with $W_2 + B_2$. So, this is this portion is basically W_2 and B_2 are coming from your second linear transformation.

[35:21] Okay? Okay? So, that's the architecture inside feedforward network. And with this one, I come to the end of today's discussion. Before I conclude, let us go through what exactly we have learned in today's session. So, we have started with what is the motivation for transformers. We have seen what are the limitations of the sequence to sequence to sequence models like RNNs and LSTMs. And we have seen the sequential computation was making the process slower, the training process slower, also parallelization was difficult, and long-range dependency was becoming a problem. Long long-range dependency was becoming a problem. So, scientists came out with the transformer architecture in a groundbreaking work in 2017 in the paper

[36:16] Attention Is All You Need. And that is where they have come out of the completely come out of the recurrences. Okay, in the attention architecture. So, this this transformer in the transformer architecture. Now in the transformer architecture we have two main parts, the encoder and the decoder. Our today's discussion was mainly on the decoder encoder part and we have seen that what are the sub layers in the encoder particularly the multi-head self-attention and the position wise feed forward network. Okay, so these are the two important things and then we have seen that there are stacks six encoder layers are stacked one after the other. And then we have understood the concept of positional encoding why it is

[37:02] required because in normal RNNs and LSTMs the words used to be given from the hidden states, you know, each and every word used to come sequentially. In the transformers case they are all given simultaneously so the fingerprint of the position positional fingerprint was missing and for that one this positional encoding was given which was injected with the vector embedding of each and every token and which then was added to the vector embedding and then we got the final embedding that is put as the input to the model. Okay, and we have done a numerical to understand how these positional encodings are done.

[37:42] Then we have seen what is the function of the various sub layers. We have seen what is the function of multi-head attention self-attention layer. It basically takes the perspective various perspectives of the same same sentence and gives a better understanding of the you know, sentence and then we have seen what is the function of the feed forward network. So we have seen the feed forward network what it actually gives much more you know, understanding deeper understanding of the sentence and then we have seen what is the function of the residual connection that is present in the encoder layer. Residual connections are added so that we don't miss any information that is that may happen when the input is going

[38:34] through several sub layers operations. Okay. So that is there and then we

have seen normalization is done and then we have seen how what is the architecture of the feedforward network. What is there inside the feedforward network? Basically, there are two linear transformations. In between them, there is a ReLU activation function. So we have seen what is the equation for that one. So with this one, we come to the end of the encoder part of transformer.

[39:03] In my next session, we will be discussing about the decoder part and the overall transformer, how it is working, how the encoder and the decoder they are coming together and bringing out the, you know, beautiful performance of the transformer. So with this one, I conclude today's session. I'll meet you all in the next session with the decoder architecture. Till that time, thank you all and happy learning.